

# Reinforcement Learning in Python

## Introduction to Reinforcement Learning

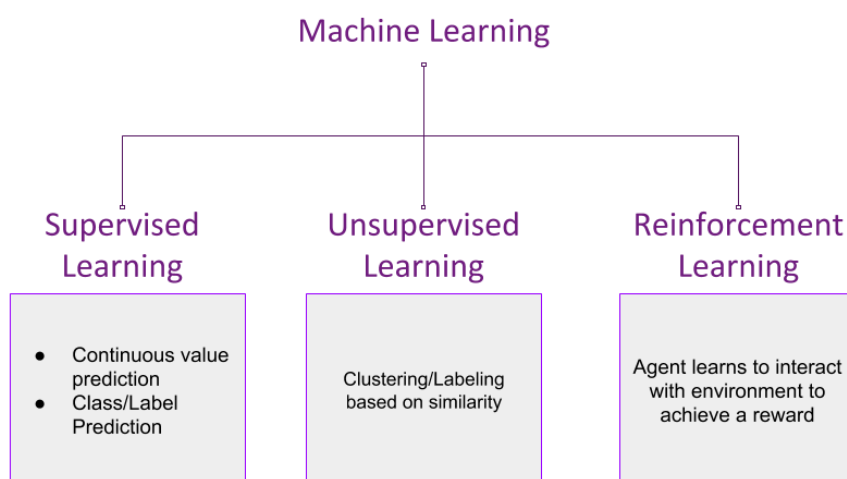
In this tutorial, you'll learn the basic concepts and terminologies of reinforcement learning. At the end of the tutorial, we'll discuss the epsilon-greedy algorithm for applying reinforcement learning based solutions.

## What is Reinforcement Learning?

Reinforcement learning in formal terms is a method of machine learning wherein the software agent learns to perform certain actions in an environment which lead it to maximum reward. It does so by exploration and exploitation of knowledge it learns by repeated trials of maximizing the reward.

## Reinforcement Learning vs. the rest

Methods of machine learning, other than reinforcement learning are as shown below -



One can conclude that while supervised learning predicts continuous ranged values or discrete labels/classes based on the training it receives from examples with provided labels or values. Unsupervised learning tries to club together samples based on their similarity and determine discrete clusters.

Reinforcement learning on the other hand, which is a subset of Unsupervised learning, performs learning very differently. It takes up the method of "cause and effect".

## Intuition to Reinforcement Learning

Let us try to understand the previously stated formal definition by means of an example.

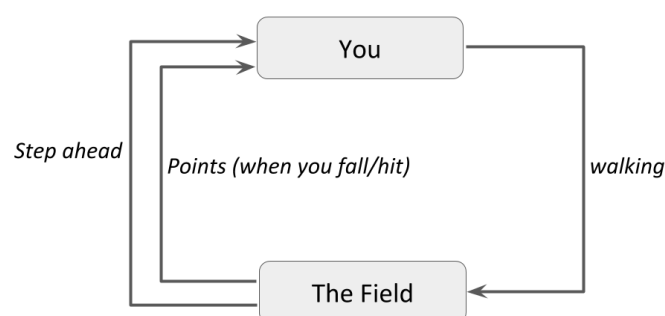
Imagine you are supposed to cross an unknown field in the middle of a pitch black night without a torch. There can be pits and stones in the field, the position of those are unfamiliar to you. There's a simple rule - if you fall into a hole or hit a rock, you must start again from your initial point.

- You start walking forward blindly, only counting the number of steps you take. After  $x$  steps, you fall into a pit. Your reward was  $x$  points since you walked that many steps.
- You start again from your initial position, but after  $x$  steps, you take a detour either left/right and again move forward. You hit a stone after  $y$  steps. This time your reward was  $y$  which is greater than  $x$ . You decide to take this path again but with more caution.
- When you start again, you make a detour after  $x$  steps, another after  $y$  steps and manage to fall into another pit after  $z$  steps. This time the reward was  $z$  points which was greater than  $y$ , and you decide that this is a good path to take again.
- You restart again, make the detours after  $x$ ,  $y$  and  $z$  steps to reach the other side of the field. Thus, you've learned to cross the field without the need of light.

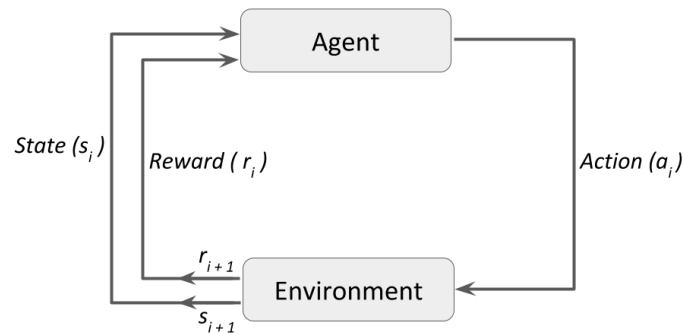
## Basic Concept and Terminology

### Insight

In the above example, you are the **agent** who is trying to walk across the field, which is the **environment**. Walking is the **action** the **agent** performs on the **environment**. The distance the **agent** walks acts as the **reward**. The **agent** tries to perform the **action** in such a way that the **reward** maximizes. This is how Reinforcement Learning works in a nutshell. The following figure puts it into a simple diagram -



And in the proper technical terms, and generalizing to fit more examples into it, the diagram becomes -



## Terminology

Some important terms related to reinforcement learning are ([These terms are taken from Steeve Huang's post on Introduction to Various Reinforcement Learning Algorithms. Part I](#))

- **Agent:** a hypothetical entity which performs actions in an environment to gain some reward.
- **Action (a):** All the possible moves that the agent can take.
- **Environment (e):** A scenario the agent must face.
- **State (s):** Current situation returned by the environment.
- **Reward (R):** An immediate return sent back from the environment to evaluate the last action by the agent.
- **Policy ( $\pi$ ):** The strategy that the agent employs to determine next action based on the current state.
- **Value (V):** The expected long-term return with discount, as opposed to the short-term reward  $R$ .  $V_\pi(s)$ , is defined as the expected long-term return of the current state  $s$  under policy  $\pi$ .
- **Q-value or action-value (Q):** Q-value is similar to Value, except that it takes an extra parameter, the current action  $a$ .  $Q_\pi(s, a)$  refers to the long-term return of the current state  $s$ , taking action  $a$  under policy  $\pi$ .

## How Reinforcement Learning Works

There are majorly three approaches to implement a reinforcement learning algorithm. They are -

- **Value Based:** in a value-based reinforcement learning method, you try to maximize a value function  $V(s)$ . As defined in the terminology previously,  $V_{\pi}(s)$  is the expected long-term return of the current state  $s$  under policy  $\pi$ . Thus,  $V(s)$  is the value of reward which the agent expects to gain in the future upon starting at that state  $s$ .

$$V_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$$

- **Policy-based:** in a policy-based reinforcement learning method, you try to come up with a policy such that the action performed at each state is optimal to gain maximum reward in the future. Here, no value function is involved. We know that the policy  $\pi$  determines the next action  $a$  at any state  $s$ . There are two types of policy-based RL methods -
  - **Deterministic:** at any state  $s$ , the same action  $a$  is produced by the policy  $\pi$ .
  - **Stochastic:** each action has a certain probability, given by the equation below -

$$\text{Stochastic Policy} : \pi(a|s) = P[A_t = a | S_t = s]$$

- **Model-Based:** in this type of reinforcement learning, you create a virtual model for each environment, and the agent learns to perform in that specific environment. Since the model differs for each environment, there is no singular solution or algorithm for this type.

## A Simple Implementation

Reinforcement Learning comes with its own classic example - the Multi-Armed Bandit problem. Never heard? No worries! Here's what it is - assume you're at a casino and in a section with some slot machines. Let's say you're at a section with 10 slot machines in a row and it says "Play for free! Max payout is 10 dollars" Each slot machine is guaranteed to give you a reward between 0 and 10 dollars. Each slot machine has a different average payout, and you must figure out which one gives the most average reward so that you can maximize your reward in the shortest time possible.

*But why is it called the Multi-Armed Bandit problem? Think of the slot machine as a one-armed (single lever) bandit (because it generally steals your money!). Multiple slot machines, thus multi-armed bandit.*

And if you're still wondering, this is what a slot machine looks like –



(Source: [Futurity](#))

One very obvious approach would be to pull the same lever every time. The probability of hitting the jackpot being very low, you'd mostly be losing money by doing this. Formally, this can be defined as a **pure exploitation** approach.

Alternatively, you could pull the lever of each slot machine in hopes that at least one of them would hit the jackpot. This is another naive approach which would give you sub-optimal returns. Formally this approach is a **pure exploration** approach.

## **$\epsilon$ (epsilon)-greedy algorithm**

One very famous approach to solving reinforcement learning problems is the  **$\epsilon$  (epsilon)-greedy** algorithm, such that, with a probability  $\epsilon$ , you will choose an action **a** at random (**exploration**), and the rest of the time (probability  $1-\epsilon$ ) you will select the best lever based on what you currently know from past plays (**exploitation**). So most of the time you play greedy, but sometimes you take some risks and choose a random lever and see what happens.

```
import numpy as np
import random
import matplotlib.pyplot as plt
%matplotlib inline
np.random.seed(5)
```

First, import the necessary libraries and modules required to implement the algorithm.

```
n = 10  
arms = np.random.rand(n)  
eps = 0.1 #probability of exploration action
```

You'll be solving the 10-armed bandit problem, hence  $n = 10$ . `arms` is a NumPy array of length  $n$  filled with random floats that can be understood as probabilities of action of that arm.

```
def reward(prob):  
    reward = 0  
    for i in range(10):  
        if random.random() < prob:  
            reward += 1  
    return reward
```

The reward functions work as such - for each arm, you run a loop of 10 iterations, and generate a random float every time. If this random number is less than the probability of that arm, you'll add a 1 to the reward. After all iterations, you'll have a value between 0 to 10.

*Why do you add a 1 only when the random number is **less** than the probability of that arm? Why not when it is more? The answer is, say you have a probability of 0.8 for an arm. You want 8 out of 10 times a positive response. That indicates that the positive response should be to the left of the probability value on the number line.*

```
#initialize memory array; has 1 row defaulted to random action index  
av = np.array([np.random.randint(0, (n+1)), 0]).reshape(1,2) #av = action-  
value  
#greedy method to select best arm based on memory array  
def bestArm(a):  
    bestArm = 0 #default to 0  
    bestMean = 0
```

```

    for u in a:
        avg = np.mean(a[np.where(a[:,0] == u[0])[:, 1]]) #calculate mean
reward for each action

        if bestMean < avg:
            bestMean = avg
            bestArm = u[0]

    return bestArm

```

The next function you define is your greedy strategy of choosing the best arm so far. This function accepts a memory array that stores the history of all actions and their rewards. It is a 2 x k matrix where each row is an index reference to your arms array (1st element), and the reward received (2nd element). For example, if a row in your memory array is [2, 8], it means that action 2 was taken (the 3rd element in our arms array) and you received a reward of 8 for taking that action.

And here is the main loop for each play. Let's play it 500 times and display a matplotlib scatter plot of the mean reward against the number of times the game is played.

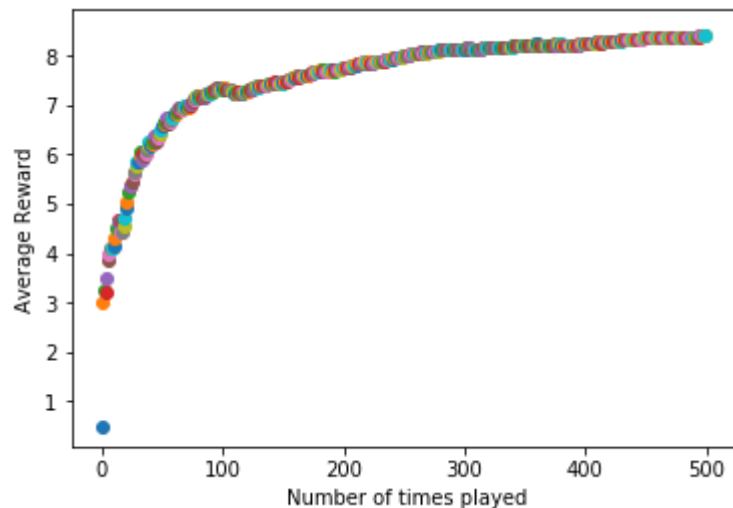
```

plt.xlabel("Number of times played")
plt.ylabel("Average Reward")
for i in range(500):
    if random.random() > eps: #greedy exploitation action
        choice = bestArm(av)
        thisAV = np.array([[choice, reward(arms[choice])]])
        av = np.concatenate((av, thisAV), axis=0)
    else: #exploration action
        choice = np.where(arms == np.random.choice(arms))[0][0]
        thisAV = np.array([[choice, reward(arms[choice])]]) #choice,
reward
        av = np.concatenate((av, thisAV), axis=0) #add to our action-
value memory array

    #calculate the mean reward

```

```
runningMean = np.mean(av[:,1])  
plt.scatter(i, runningMean)
```



As expected, your agent learns to choose the arm which gives it the maximum average reward after several iterations of gameplay. Thus, you've implemented a straightforward reinforcement learning algorithm to solve the Multi-Arm Bandit problem.

## Conclusion

Reinforcement learning is becoming more popular today due to its broad applicability to solving problems relating to real-world scenarios. It has found significant applications in the fields such as -

- **Game Theory and Multi-Agent Interaction** - reinforcement learning has been used extensively to enable game playing by software. A recent example would be Google's [DeepMind](#) which was able to defeat the world's highest ranked Go player and later, the highest rated Chess program Komodo.
- **Robotics** - robots have often relied upon reinforcement learning to perform better in the environment they are presented with. Reinforcement learning comes with the benefit of being a play and forget solution for robots which may have to face unknown or continually changing environments. One well-known example is the [Learning Robots by Google X](#) project.



- **Vehicle navigation** - vehicles learn to navigate the track better as they make re-runs on the track. A proof of concept is presented in [this video](#) while a real-life world example was presented at the O'Reilly AI Conference [in this video](#).
- **Industrial Logistics** - industry tasks are often automated with the help of reinforcement learning. The software agent facilitating it gets better at its task as time passes. [BonsAI](#) is a startup working to bring such AI to the industries.

There are many more reinforcement learning algorithms to explore and quite a few that perform better than the simple algorithm we've demonstrated above. Do check out the References and Links for some quick places to start with your exploration and mix it with the exploitation of this tutorial to reinforce your learning of the topic.

## References

- Sutton, Richard S. and Barto, Andrew G., [Reinforcement Learning: An Introduction](#), MIT Press, 1998
- [The Reinforcement Learning Repository](#), University of Massachusetts, Amherst
- [Wikipedia article on Reinforcement Learning](#)
- [A Beginners Guide to Deep Reinforcement Learning](#)

## Links

If you still have doubts or wish to read up more about reinforcement learning, these links can be a great starting point.

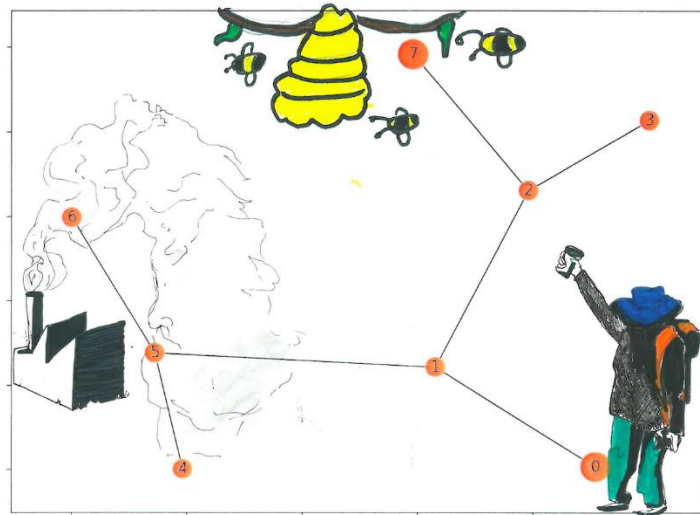
- [A Glossary of terms in Reinforcement Learning](#)
- [Bibliography on Reinforcement Learning](#)
- [David J. Finton's Reinforcement Learning Page](#)
- [Stanford University Andrew Ng Lecture on Reinforcement Learning](#)

# Reinforcement Learning - A Simple Python Example and a Step Closer to AI with Assisted Q-Learning

Practical walkthroughs on machine learning, data exploration and finding insight.

## Resources

- [YouTube Companion Video](#)



Q-learning is a model-free reinforcement learning technique. Specifically, Q-learning can be used to find an optimal action-selection policy for any given (finite) Markov decision process (MDP). [Q-learning - Wikipedia](#)

Machine learning is assumed to be either supervised or unsupervised, but a recent new-comer broke the status-quo - reinforcement learning. Supervised and unsupervised approaches require data to model, not reinforcement learning! That's right, it can explore space with a handful of instructions, analyse its surroundings one step at a time, and build data as it goes along for modelling.

In this walk-through, we'll use Q-learning to find the shortest path between two areas. It has the ability to embark on a journey with no knowledge of what to do next. This approach requires constant trial and error as it collects data about its surroundings and figures out how to accomplish its goal. This opens up interesting possibilities, what about recording additional information, like environmental details along the way that it may not fully understand until after it reaches its goal? And once reached, could it review that additional data to determine if any of it would have helped it reach its goal faster?

## A VERY Simple Python Q-learning Example

But let's first look at a very simple python implementation of q-learning - no easy feat as most examples on the Internet are too complicated for newcomers. The code is heavily borrowed from Mic's great blog post [Getting AI smarter with Q-learning: a simple first step in Python](#).

```
import numpy as np
import pylab as plt

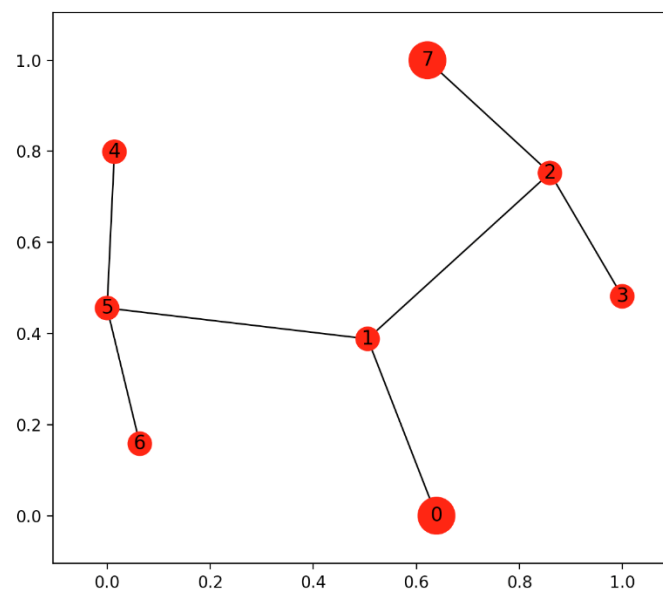
# map cell to cell, add circular cell to goal point
points_list = [(0,1), (1,5), (5,6), (5,4), (1,2), (2,3), (2,7)]
```

We create a points-list map that represents each direction our bot can take. Using this format allows us to easily create complex graphs but also easily visualize everything with [networkx](#) graphs.

Our starting point is **0**, our goal point is **7**.

```
goal = 7

import networkx as nx
G=nx.Graph()
G.add_edges_from(points_list)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G,pos)
nx.draw_networkx_edges(G,pos)
nx.draw_networkx_labels(G,pos)
plt.show()
```



The map shows that point **0** is where our bot will start its journey and point **7** is its final goal. The extra added points and false paths are the obstacles the bot will have to contend with. If you look at the top image, we can weave a story into this search - our bot is looking for honey, it is trying to find the hive and avoid the factory (the storyline will make sense in the second half of the article).

We then create the rewards graph - this is the matrix version of our list of points maps. We initialize the matrix to be the height and width of our points list (8 in this example) and initialize all values to **-1**:

```
# how many points in graph? x points
MATRIX_SIZE = 8

# create matrix x*y
R = np.matrix(np.ones(shape=(MATRIX_SIZE, MATRIX_SIZE)))
R *= -1
```

We then change the values to be 0 if it is a viable path and 100 if it is a goal path (for more on this topic, see Mnemosyne\_studio's great tutorial: [Deep Q Learning for Video Games - The Math of Intelligence #9](#))

```
# assign zeros to paths and 100 to goal-reaching point
for point in points_list:
    print(point)
    if point[1] == goal:
        R[point] = 100
    else:
        R[point] = 0

    if point[0] == goal:
        R[point[:-1]] = 100
    else:
        # reverse of point
        R[point[::-1]] = 0

# add goal point round trip
R[goal,goal]= 100

R
matrix([[ -1.,    0.,   -1.,   -1.,   -1.,   -1.,   -1.,   -1.],
 [    0.,   -1.,    0.,   -1.,   -1.,    0.,   -1.,   -1.],
 [   -1.,    0.,   -1.,    0.,   -1.,   -1.,   -1.,  100.],
 [   -1.,   -1.,    0.,   -1.,   -1.,   -1.,   -1.,   -1.],
 [   -1.,   -1.,   -1.,   -1.,   -1.,    0.,   -1.,   -1.],
 [   -1.,    0.,   -1.,   -1.,    0.,   -1.,    0.,   -1.],
 [   -1.,   -1.,   -1.,   -1.,   -1.,    0.,   -1.,   -1.],
 [   -1.,   -1.,    0.,   -1.,   -1.,   -1.,   -1.,  100.]])
```

To read the above matrix, the y-axis is the state or where your bot is currently located, and the x-axis is your possible next actions. We then build our Q-learning matrix which will hold all the lessons

learned from our bot. The Q-learning model uses a transitional rule formula and gamma is the learning parameter (see [Deep Q Learning for Video Games - The Math of Intelligence #9](#) for more details). The rest of this example is mostly copied from Mic's blog post [Getting AI smarter with Q-learning: a simple first step in Python](#).

```
Q = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))

# learning parameter
gamma = 0.8

initial_state = 1

def available_actions(state):
    current_state_row = R[state,]
    av_act = np.where(current_state_row >= 0)[1]
    return av_act

available_act = available_actions(initial_state)

def sample_next_action(available_actions_range):
    next_action = int(np.random.choice(available_act,1))
    return next_action

action = sample_next_action(available_act)

def update(current_state, action, gamma):

    max_index = np.where(Q[action,] == np.max(Q[action,]))[1]

    if max_index.shape[0] > 1:
        max_index = int(np.random.choice(max_index, size = 1))
    else:
        max_index = int(max_index)
    max_value = Q[action, max_index]

    Q[current_state, action] = R[current_state, action] + gamma * max_value
    print('max_value', R[current_state, action] + gamma * max_value)

    if (np.max(Q) > 0):
        return(np.sum(Q/np.max(Q)*100))
    else:
        return (0)

update(initial_state, action, gamma)
```

We keep following Mic's blog and run the training and testing functions that will run the update function 700 times allowing the Q-learning model to figure out the most efficient path:

```
# Training
scores = []
for i in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
    available_act = available_actions(current_state)
```

```

    action = sample_next_action(available_act)
    score = update(current_state, action, gamma)
    scores.append(score)
    print ('Score:', str(score))

print("Trained Q matrix:")
print(Q/np.max(Q)*100)

# Testing
current_state = 0
steps = [current_state]

while current_state != 7:

    next_step_index = np.where(Q[current_state,] ==
np.max(Q[current_state,]))[1]

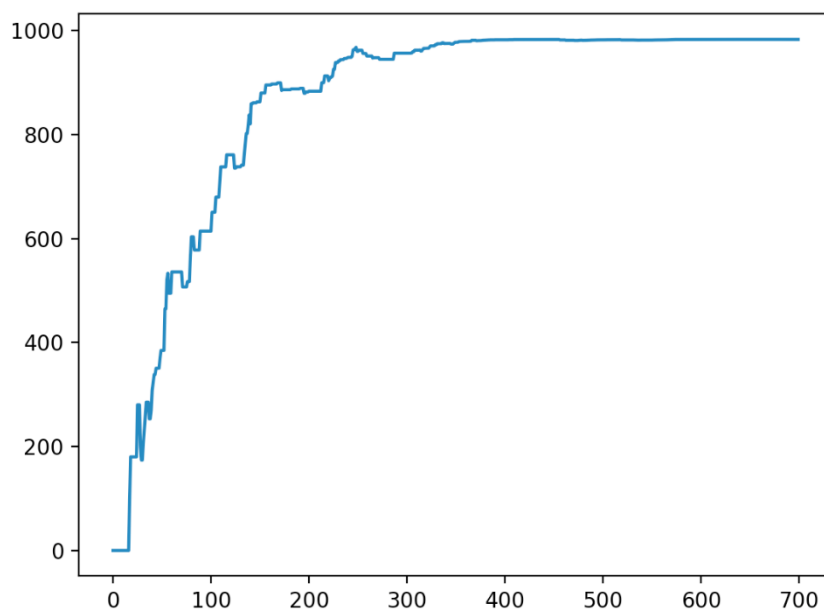
    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index, size = 1))
    else:
        next_step_index = int(next_step_index)

    steps.append(next_step_index)
    current_state = next_step_index

print("Most efficient path:")
print(steps)

plt.plot(scores)
plt.show()
Most efficient path:
[0, 1, 2, 7]

```



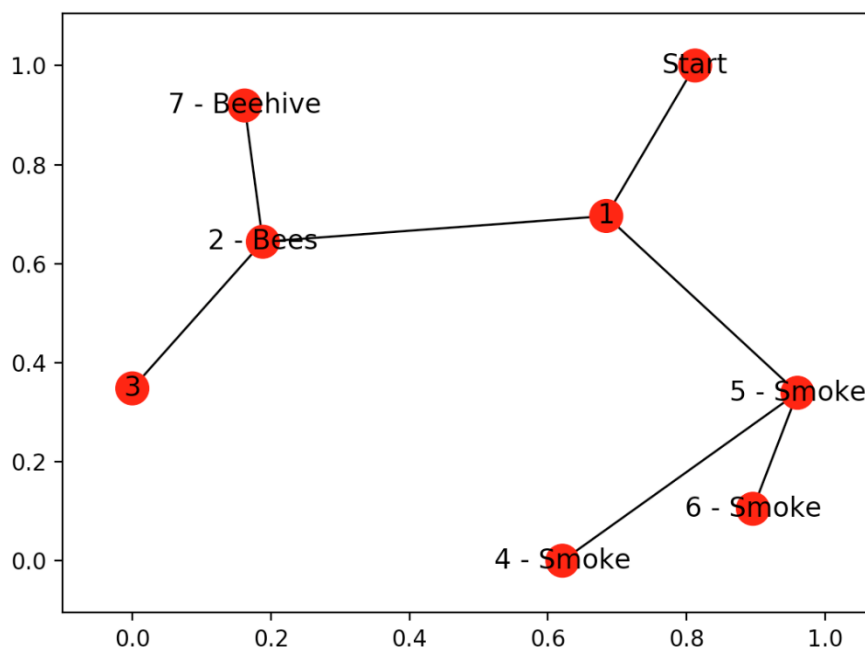
We see that the model did correctly find the most efficient path from the starting node **0** to the goal node **7** and took around 400 iterations to converge to a solution.

## Version 2.0, with Environmental Details

Now let's take this a step further, look at the top image again, notice how the factory is surrounded by smoke and the hive, by bees. Let's assume that bees don't like smoke or factories, thus there will never be a hive or bees around smoke. What if our bot could record those environmental factors and turn them into actionable insight? Whenever the bot finds smoke, it can turn around immediately instead of continuing to the factory, whenever it finds bees, it can stick around and assume the hive is close.

```
bees = [2]
smoke = [4,5,6]

G=nx.Graph()
G.add_edges_from(points_list)
mapping={0:'Start', 1:'1', 2:'2 - Bees', 3:'3', 4:'4 - Smoke', 5:'5 - Smoke',
6:'6 - Smoke', 7:'7 - Beehive'}
H=nx.relabel_nodes(G,mapping)
pos = nx.spring_layout(H)
nx.draw_networkx_nodes(H,pos, node_size=[200,200,200,200,200,200,200,200])
nx.draw_networkx_edges(H,pos)
nx.draw_networkx_labels(H,pos)
plt.show()
```



We assign node **2** as having bees and nodes **4,5,6** as having smoke. Our Q-learning bot doesn't know yet that there are bees or smoke there nor does it know that bees are good and smoke bad in finding

hives. The bot needs to do another run like we just did, but this time it needs to collect environmental factors.

Here is the new **update** function with the capability of updating the Q-learning scores when it finds either bees or smoke.

```
# re-initialize the matrices for new run
Q = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))

enviro_bees = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))
enviro_smoke = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))

initial_state = 1

def available_actions(state):
    current_state_row = R[state,]
    av_act = np.where(current_state_row >= 0)[1]
    return av_act

def sample_next_action(available_actions_range):
    next_action = int(np.random.choice(available_act,1))
    return next_action

def collect_environmental_data(action):
    found = []
    if action in bees:
        found.append('b')

    if action in smoke:
        found.append('s')
    return (found)

available_act = available_actions(initial_state)

action = sample_next_action(available_act)

def update(current_state, action, gamma):
    max_index = np.where(Q[action,] == np.max(Q[action,]))[1]

    if max_index.shape[0] > 1:
        max_index = int(np.random.choice(max_index, size = 1))
    else:
        max_index = int(max_index)
    max_value = Q[action, max_index]

    Q[current_state, action] = R[current_state, action] + gamma * max_value
    print('max_value', R[current_state, action] + gamma * max_value)

    environment = collect_environmental_data(action)
    if 'b' in environment:
        enviro_bees[current_state, action] += 1

    if 's' in environment:
        enviro_smoke[current_state, action] += 1
```



```

if (np.max(Q) > 0):
    return(np.sum(Q/np.max(Q)*100))
else:
    return (0)

update(initial_state,action,gamma)

scores = []
for i in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
    available_act = available_actions(current_state)
    action = sample_next_action(available_act)
    score = update(current_state,action,gamma)

# print environmental matrices
print('Bees Found')
print(enviro_bees)
print('Smoke Found')
print(enviro_smoke)
Bees Found
[[ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. 21.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. 93.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. 41.  0.  0.  0.  0.  0.]]

Smoke Found
[[ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. 30.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. 90.  0.  0.]
 [ 0.  0.  0.  0. 23.  0. 29.  0.]
 [ 0.  0.  0.  0.  0. 92.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]]

```

The environmental matrices show how many bees and smoke the bot found during its journey while searching for the most efficient path to the hive. To make this walk-through simpler, I am assuming two things - we modelled the environmental data and found out that the **bees** have a positive coefficient on finding hives, and **smoke**, a negative one. And we are going to reuse the environmental matrix already mapped out for our landscape, a more realistic approach would be to dynamically look at a new environment and assign environmental biases as they are encountered.

```

Q = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))

# subtract bees with smoke, this gives smoke a negative effect
enviro_matrix = enviro_bees - enviro_smoke

# Get available actions in the current state

```

```

available_act = available_actions(initial_state)

# Sample next action to be performed
action = sample_next_action(available_act)

# This function updates the Q matrix according to the path selected and the Q
# learning algorithm
def update(current_state, action, gamma):

    max_index = np.where(Q[action,] == np.max(Q[action,]))[1]

    if max_index.shape[0] > 1:
        max_index = int(np.random.choice(max_index, size = 1))
    else:
        max_index = int(max_index)
    max_value = Q[action, max_index]

    Q[current_state, action] = R[current_state, action] + gamma * max_value
    print('max_value', R[current_state, action] + gamma * max_value)

    environment = collect_environmental_data(action)
    if 'b' in environment:
        enviro_matrix[current_state, action] += 1
    if 's' in environment:
        enviro_matrix[current_state, action] -= 1

    return(np.sum(Q/np.max(Q)*100))

update(initial_state,action,gamma)

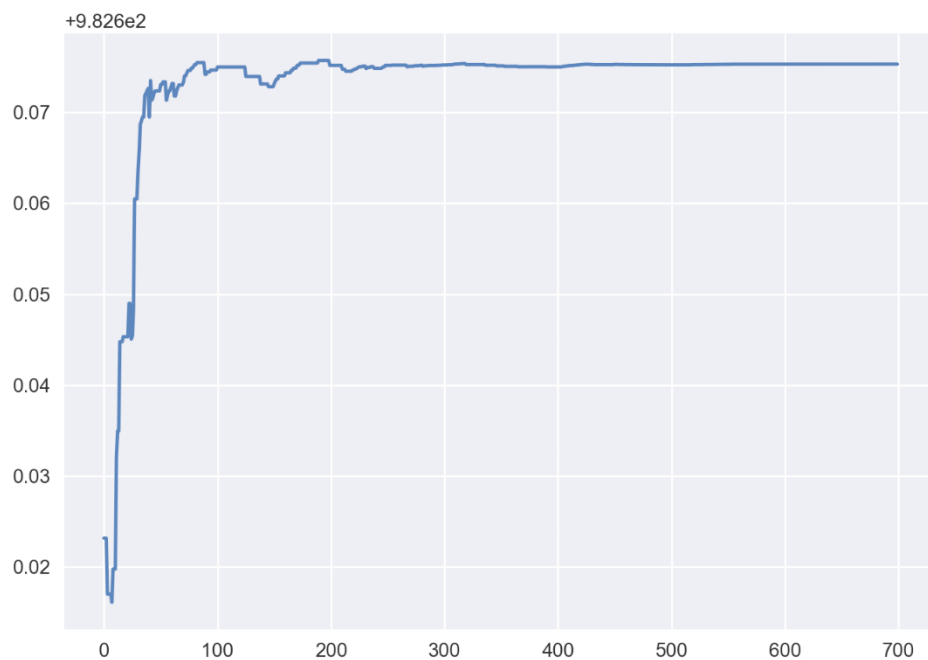
enviro_matrix_snap = enviro_matrix.copy()

def available_actions_with_enviro_help(state):
    current_state_row = R[state,]
    av_act = np.where(current_state_row >= 0)[1]
    # if there are multiple routes, dis-favor anything negative
    env_pos_row = enviro_matrix_snap[state,av_act]
    if (np.sum(env_pos_row < 0)):
        # can we remove the negative directions from av_act?
        temp_av_act = av_act[np.array(env_pos_row)[0]>=0]
        if len(temp_av_act) > 0:
            print('going from:',av_act)
            print('to:',temp_av_act)
            av_act = temp_av_act
    return av_act

# Training
scores = []
for i in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
    available_act = available_actions_with_enviro_help(current_state)
    action = sample_next_action(available_act)
    score = update(current_state,action,gamma)
    scores.append(score)
    print ('Score:', str(score))

```

```
plt.plot(scores)
plt.show()
```



We see that the bot converges in less tries, say around 100 less, than our original model. This isn't meant to be a controlled environment to compare both approaches, instead it's about triggering thoughts on different ways of applying reinforced learning for discovery.